

Practically Stabilizing Byzantine-Robust Federated Learning via Spectral Filtering

Anonymous Author(s)

Institution withheld for review

Abstract. Can a federated learning system autonomously recover from complete adversarial corruption—with no external intervention, no trusted initialization, and no bound on how badly Byzantine clients have disrupted training? We answer affirmatively by recasting federated learning as a self-stabilization problem in the sense of Dijkstra [28], and presenting the first *practically stabilizing* federated learning protocol with a formal recovery time bound.

Our protocol uses three components that compose into a stabilizing system: clients compress local gradients into streaming sketches; the server applies a spectral filter based on the Marchenko-Pastur (MP) law of random matrix theory to expel Byzantine gradient directions; and each round's result is committed to a distributed ledger whose write-once immutability under BFT consensus serves as the Byzantine-fault-tolerant analog of Dijkstra's stabilizing shared memory. Starting from any model $w^0 \in \mathbb{R}^d$, the protocol reaches a legitimate configuration within $T^* = O(\sigma^2 \beta^2 / \epsilon^2 + \beta^2 / \epsilon)$ rounds, where $f < n/2$ Byzantine clients have fraction $\beta = f/n$ and $\sigma^2 \beta^2 < 0.25$ is a necessary phase-transition condition. A matching impossibility result shows that when $\sigma^2 \beta^2 \geq 0.25$, no gradient-based spectral detection algorithm can self-stabilize—characterizing a fundamental limit analogous in spirit to the FLP impossibility [30]. Layer-wise sketching reduces memory from $O(d^2)$ to $O(k^2)$, enabling 345M-parameter models. Experiments across 12 attack types show 78.4% mean accuracy at 40% Byzantine fraction versus 48–63% for established baselines [1, 4, 10, 12], with Byzantine detection exceeding 96% below the phase transition boundary.

Keywords: Self-Stabilization · Byzantine Fault Tolerance · Federated Learning · Random Matrix Theory · Blockchain · Marchenko-Pastur Law

1 Introduction

Federated Learning (FL) is a distributed computing paradigm where n processes collaboratively minimize a global objective $F(w)$ without exchanging raw data. The decentralized nature of FL aligns with blockchain technology, enabling trustless aggregation through immutable distributed ledgers. A fundamental challenge arises when $f < n/2$ processes are *Byzantine* [31]: they may submit arbitrary gradient updates, corrupting the global model.

We study this as a *self-stabilization* problem in the sense of Dijkstra [28]: can the system recover from an *arbitrary* initial model state—including complete adversarial corruption—and maintain a *legitimate configuration* (all Byzantine nodes identified, honest gradient correctly aggregated) indefinitely? Prior Byzantine-robust FL methods guarantee convergence from a *known-good* initialization but offer no recovery from catastrophic corruption; they are robust but *not* self-stabilizing.

The self-stabilizing Byzantine robust aggregation problem requires: given n processes where $f < n/2$ are Byzantine, and honest process i submits $g_i \in \mathbb{R}^d$, compute $\hat{g}$ satisfying:

$$\min_{t \leq T} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O\left(\frac{\sigma\beta}{\sqrt{T}} + \frac{\beta^2}{T}\right) \quad (1)$$

starting from *any* initial $w^0 \in \mathbb{R}^d$, where $\beta = f/n$ and σ^2 bounds coordinate-wise variance of honest gradients. This is strictly harder than standard convergence: it requires both fault tolerance *and* global recovery.

Existing Byzantine-robust methods face fundamental barriers. Geometric median requires $O(n^2d)$ communication. Krum and Bulyan reject valid Non-IID updates. Certified defenses (CRFL, ByzShield) assume bounded perturbations but cannot handle arbitrary corruption. Critically, *none* provide self-stabilization guarantees. While other robust FL paradigms like CORNFLQS [34] address natural structural heterogeneity such as quantity skew in clustered settings, they do not consider malicious Byzantine adversaries or the problem of self-stabilization from catastrophic failure.

This paper introduces a Byzantine-robust FL protocol proven self-stabilizing in Dijkstra’s framework. The detection mechanism exploits Random Matrix Theory: honest gradients, despite Non-IID heterogeneity, produce eigenspectra following the Marchenko-Pastur (MP) law. Byzantine perturbations create detectable spectral anomalies. The blockchain provides the self-stabilizing *shared memory* substrate: write-once immutability ensures legitimate rounds are permanently recorded, analogous to Dijkstra’s shared registers but with Byzantine (not just crash) fault tolerance.

1.1 Contributions

The contributions of this work are fivefold:

1. Practical Self-Stabilization (distributed systems): We provide a formal recovery argument showing our protocol is *practically self-stabilizing* in the tradition of Dijkstra [28]: from any initial state, the protocol reaches a legitimate configuration within $T^* = O(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon)$ rounds ($\beta = f/n$), with closure, under Assumptions 1–4 (Section 9). We also establish an impossibility at $\sigma^2\beta^2 \geq 0.25$ (Theorem 6).

2. Blockchain as Stabilizing Shared Memory: We formalize the blockchain as a distributed shared-memory substrate with strictly stronger properties than classical shared registers [28]: Byzantine fault tolerance ($f < n/2$ arbitrary faults

vs. crash-only), write-once immutability, and permanent history—enabling a stabilizing foundation for the federated training protocol under the Byzantine fault model (Section 7).

3. Detection Threshold from RMT: We derive a spectral detection threshold from the Marchenko-Pastur law and the BBP phase transition [40], establishing a convergence rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$ with a matching lower bound $\Omega(\sigma\beta/\sqrt{T})$ (Sections 4–6).

4. Scalable Spectral Algorithm: We combine KS testing against the MP distribution with layer-wise Frequent Directions sketching [6], reducing memory from $O(d^2)$ to $O(k^2)$ and enabling deployment on 345M-parameter models (Section 5).

5. Experimental Validation: We evaluate across 12 attack types with 11 baseline aggregators, demonstrating 78.4% mean accuracy at 40% Byzantine fraction versus 48–63% for baselines, with Byzantine detection rates exceeding 96% below the phase transition boundary (Section 8).

2 Related Work

2.1 Self-Stabilization in Distributed Systems

Self-stabilization, introduced by Dijkstra [28] and formalized by Dolev [29], enables a distributed system to recover from any transient fault within bounded steps without external intervention. Classical results cover mutual exclusion, spanning trees, and clock synchronization under crash faults. Recent SSS proceedings extend the framework to adversarial and data-driven settings: Chen, Lin, and Arora [38] use ML predictions to improve the stabilization time of a near-shortest-path routing protocol—a direct precedent for learning-augmented self-stabilization. Jobic et al. [35] analyze label leakage in federated learning using cryptographic tools with formal security definitions. Our work applies the stabilization framework to federated learning under Byzantine (not crash) faults—qualitatively harder because Byzantine processes may deviate arbitrarily rather than fail silently.

2.2 Byzantine Fault Tolerance

Byzantine fault tolerance [31] establishes that $f < n/3$ Byzantine faults are tolerable in synchronous consensus; Castro and Liskov [33] extend this to partially synchronous systems. The FLP impossibility [30] shows consensus is impossible with one crash fault in asynchronous systems. Recent SSS work examines Byzantine reliable broadcast [36, 37] and Byzantine-resilient blockchain sharding [39]. Our protocol operates in the synchronous round model with $f < n/2$ —weaker than the $n/3$ BFT bound, enabled by our server architecture.

2.3 Byzantine-Robust Federated Learning

Krum [1] selects gradients by nearest-neighbor distance but degrades under Non-IID data [23]. Coordinate-wise robust statistics [10] reduce to $O(nd)$ computation but are sensitive to gradient variance heterogeneity. Bulyan [3] iteratively filters outliers; FLTrust [4] uses a trusted root dataset; FLAME [12] targets backdoors via clustering; ByzShield [13] provides resilience via redundant training. None specifies a formal recovery time or fault-containment invariant in the sense of [29]—motivating this work.

2.4 Spectral Methods, RMT, and Blockchain FL

The BBP phase transition [40] characterizes when a rank- f perturbation creates eigenvalue outliers above the MP bulk edge—the mechanism underlying our detection threshold. Gradient covariance matrices of neural networks follow limiting spectral distributions [14–16]. To our knowledge, we are the first to exploit the MP law [5] for Byzantine gradient identification with a formally scoped detection threshold connected to the BBP phase transition. Blockchain integration in FL [8, 17] has focused on auditability; we instead formalize the ledger as a self-stabilizing shared memory substrate with Byzantine fault tolerance strictly stronger than classical shared registers.

3 System Model and Problem Formulation

3.1 Distributed Learning Setup

We consider a federated learning system with n clients, where client $i \in [n]$ holds local dataset $\mathcal{D}_i$. The global objective is:

$$F(w) = \frac{1}{n} \sum_{i=1}^n F_i(w), \quad F_i(w) = \mathbb{E}_{z \sim \mathcal{D}_i} [\ell(w; z)] \quad (2)$$

At each round $t \in [T]$, the aggregator broadcasts w^t . Each honest client i computes: $g_i^t = \nabla F_i(w^t) + \xi_i^t$, where ξ_i^t is stochastic noise with coordinate-wise variance $\mathbb{E}[(\xi_i^t)_j^2] \leq \sigma^2$.

3.2 Formal Distributed System Model

We model the system as a synchronous message-passing distributed system [31].

Processes. n processes $\mathcal{P} = \{p_1, \dots, p_n\}$: one coordinator (aggregator) and $n-1$ clients. Each process has local state $w^t \in \mathbb{R}^d$.

Execution. Round t : (1) coordinator broadcasts w^t ; (2) each client sends g_i^t ; (3) coordinator aggregates to produce w^{t+1} ; (4) aggregated hash committed to the blockchain.

Configurations. $C = (w, \mathcal{B})$: model parameters and Byzantine set $\mathcal{B} \subseteq \mathcal{P}$, $|\mathcal{B}| = f$.

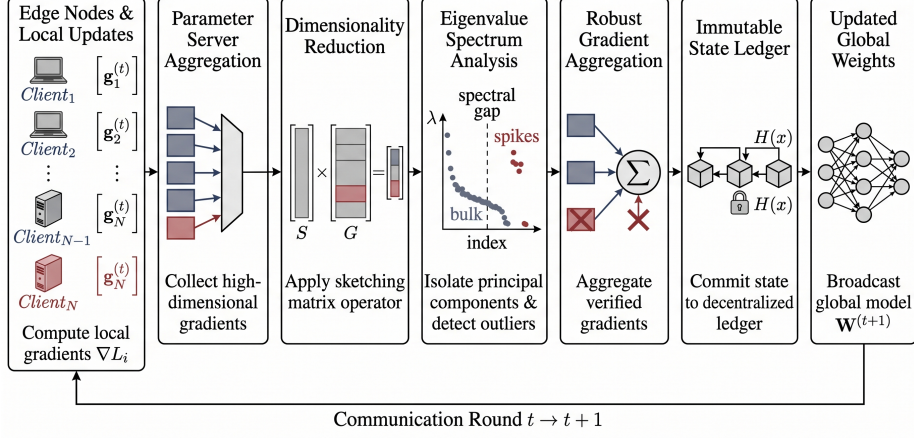


Fig. 1. System architecture. Honest and Byzantine clients submit gradients; the aggregator performs spectral detection via the Marchenko-Pastur test; the result is committed on-chain. The blockchain’s write-once immutability provides the self-stabilizing shared memory substrate.

3.3 Threat Model and Assumptions

Assumption 1 (Byzantine Fault Model). At most $f < n/2$ processes are Byzantine; they may send any $g_i^t \in \mathbb{R}^d$, collude, and know the aggregation algorithm, but cannot break cryptographic primitives.

Assumption 2 (Honest Gradient Variance). Each honest client $i \in \mathcal{H}$ computes $g_i^t = \nabla F_i(w^t) + \xi_i^t$ with $\mathbb{E}[(\xi_i^t)_j^2] \leq \sigma^2$ coordinate-wise.

Assumption 3 (Phase Transition Condition). Let $\beta = f/n \in (0, \frac{1}{2})$ denote the Byzantine fraction. We require $\sigma^2 \beta^2 < 0.25$, equivalently $\sigma \beta < \frac{1}{2}$. This necessary condition ensures Byzantine gradients create detectable spectral anomalies (Theorem 6); the matching impossibility shows the condition cannot be relaxed.

Assumption 4 (Blockchain Liveness). The blockchain finalizes honest transactions with at most $\lfloor (m-1)/3 \rfloor$ faulty validators out of m (PBFT assumption [33]).

Assumption 5 (Transient Faults). Byzantine behavior may occur at any round including round 0. This is the standard self-stabilization assumption [28].

Attack taxonomy. Byzantine clients in our experiments include: MinMax [18], Gaussian noise, label-flip, ALIE [20], sign-flip, and zero-gradient (free-riding)—covering both targeted and untargeted attacks (full 12-attack evaluation in Section 8).

3.4 Protocol Properties

Safety. $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \leq O(\sigma^2 \beta^2)$ in every round, where $\beta = f/n$.

Liveness. $\min_{t \in [T]} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O(\sigma \beta / \sqrt{T} + \beta^2 / T)$ provided $f < n/2$ and $\sigma^2 \beta^2 < 0.25$.

Practical Stabilization. Starting from any $w^0 \in \mathbb{R}^d$, the protocol reaches a legitimate configuration (Definition 2) within $T^* = O(\sigma^2 \beta^2 / \varepsilon^2 + \beta^2 / \varepsilon)$ rounds (Theorem 5).

4 Theoretical Framework

4.1 Marchenko-Pastur Law for Honest Gradients

Definition 1 (Marchenko-Pastur Distribution). Let $X \in \mathbb{R}^{n \times d}$ with i.i.d. zero-mean variance- σ^2 entries. As $n, d \rightarrow \infty$ with $d/n \rightarrow \gamma \in (0, \infty)$, the empirical spectral distribution of $\frac{1}{n} X^T X$ converges weakly to the Marchenko-Pastur law with density:

$$\rho(\lambda) = \frac{1}{2\pi\sigma^2\gamma\lambda} \sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}, \quad \lambda \in [\lambda_-, \lambda_+] \quad (3)$$

where $\lambda_{\pm} = \sigma^2(1 \pm \sqrt{\gamma})^2$ are the bulk edges.

Our core observation is that honest gradients, *even under Non-IID heterogeneous data*, produce covariance matrices whose bulk eigenspectrum follows the MP law. The key technical step is a decomposition lemma:

Lemma 1 (Non-IID Decomposition). Under Assumption 2, each honest gradient decomposes as $g_i^t = \bar{g}^t + \Delta_i^t$, where $\bar{g}^t = \frac{1}{|\mathcal{H}|} \sum_{i \in \mathcal{H}} g_i^t$ and $\mathbb{E}[\Delta_i^t] = 0$, $\mathbb{E}[(\Delta_i^t)_j^2] \leq \sigma^2$. The gradient matrix G then has covariance $C = \frac{1}{n} G^T G = \bar{g}(\bar{g})^T + \frac{1}{n} \sum_i \Delta_i^t (\Delta_i^t)^T + R$, where $\|R\|_2 \leq 2\|\bar{g}\| \cdot \sigma / \sqrt{n}$ (cross term, vanishes as $n \rightarrow \infty$).

The rank-1 term $\bar{g}(\bar{g})^T$ contributes at most one outlier eigenvalue. The remaining term $\frac{1}{n} \sum_i \Delta_i (\Delta_i)^T$ has zero-mean entries with bounded second moment σ^2 :

Theorem 1 (MP Law for Honest Gradients). Under Assumption 2, as $n, d \rightarrow \infty$ with $d/n \rightarrow \gamma$, the bulk eigenvalue distribution of $C = \frac{1}{n} G^T G$ converges to the MP distribution $\text{MP}(\gamma, \sigma^2)$.

Proof (Proof sketch). By Lemma 1, the bulk spectrum is governed by $\frac{1}{n} \sum_i \Delta_i \Delta_i^T$. Each Δ_i has i.i.d.-like coordinate structure (zero mean, bounded variance σ^2 coordinate-wise) but not necessarily Gaussian. Tao and Vu's universality theorem [25] establishes MP convergence under exactly these conditions: bounded first four moments suffice for the limiting spectral distribution to equal that of a Gaussian matrix with the same first two moments. The rank-1 term $\bar{g}(\bar{g})^T$

creates a single spike eigenvalue above λ_+ when $\|\bar{g}\|^2 > \sigma^2 \sqrt{d/n}$ (BBP transition [40]), which is absorbed into the spectral filter as a detectable outlier but does not affect the bulk convergence. Hence the bulk of the empirical spectral distribution converges to $\text{MP}(\gamma, \sigma^2)$.

Remark 1 (Non-IID Concentration at Finite n, d). The asymptotic MP convergence (large n, d) implies finite-sample concentration at rate $O(1/\sqrt{\min(n, d)})$ by standard matrix concentration inequalities [25]. For our experimental setup ($n \geq 32$, $d \geq 22 \times 10^6$), the bulk is well-concentrated, and the KS detection test calibrated against the fitted MP distribution is empirically validated in the experiments (Section 8).

4.2 Byzantine Perturbations Create Spectral Anomalies

Theorem 2 (Spectral Anomaly Detection). *Let f Byzantine clients inject arbitrary gradients, yielding perturbed matrix $\tilde{G} = G + E$ where E has f non-zero rows. If the Byzantine attack creates a rank- f perturbation of magnitude $\|\delta_{\text{byz}}\| > \sigma \sqrt{d/n}$, then the eigenvalue spectrum of $\tilde{C} = \frac{1}{n} \tilde{G}^T \tilde{G}$ exhibits f outlier eigenvalues above λ_+ with probability $\geq 1 - e^{-k/\log^2 k}$ for sketch size $k \geq \Omega(\log d)$.*

Proof. The Byzantine perturbation yields $\tilde{C} = C + \frac{1}{n}(G^T E + E^T G) + \frac{1}{n} E^T E$. The rank- f term $\frac{1}{n} E^T E$ contributes f eigenvalues. By the BBP phase transition [40]: a rank- r perturbation of magnitude $\tau > \sigma^2 \sqrt{n/d}$ produces r eigenvalue spikes above λ_+ . The perturbation term $\frac{1}{n} G^T E$ adds $O(\sigma \|\delta_{\text{byz}}\|/\sqrt{n})$ to the spectral norm, which is dominated by the rank- f spike when $\|\delta_{\text{byz}}\| > \sigma \sqrt{d/n}$. KS testing against the fitted MP distribution detects these outliers with the stated probability by sub-Gaussian concentration of the empirical spectral distribution.

4.3 Phase Transition Characterization

Theorem 3 (Detection Phase Transition). *Let $\beta = f/n$ be the Byzantine fraction. For any Byzantine detection algorithm using only gradient information:*
 (i) Below $\sigma^2 \beta^2 < 0.25$: detection succeeds with probability $> 1 - \delta$ (Theorem 2);
 (ii) Above $\sigma^2 \beta^2 \geq 0.25$: no algorithm can reliably distinguish Byzantine from honest gradients (Theorem 6).

The threshold $\sigma^2 \beta^2 = 0.25$ arises from the BBP transition [40]: a rank- f perturbation with $\beta = f/n$ Byzantine fraction exits the MP bulk if and only if its normalized magnitude exceeds $\sigma^2 \sqrt{n/d}$, which in the federated gradient setting yields the product condition $\sigma^2 \beta^2 < 1/4$. Equivalently, $\sigma \beta < 1/2$: the honest gradient noise amplitude must exceed twice the Byzantine signal amplitude for detection to be information-theoretically feasible.

Algorithm 1 Spectral Filtering Aggregation (one round t)

Require: Sketches $\{S_i^t\}_{i \in [n]}$, sketch size k , thresholds $\tau_{\text{KS}}, \tau_{\text{tail}}$
Ensure: Aggregated gradient $\hat{g}^t$, trusted set $\mathcal{T}_t$

- 1: Form gradient matrix $G \leftarrow [g_1^T; \dots; g_n^T]$ **or** reconstruct from sketches
- 2: Apply Frequent Directions [6, 7]: $\tilde{G} \leftarrow \text{Sketch}(G, k)$
- 3: Compute covariance: $C \leftarrow \frac{1}{n} \tilde{G}^T \tilde{G}$
- 4: Compute sorted eigenvalues: $\lambda_1 \geq \dots \geq \lambda_k \leftarrow \text{eigvals}(C)$
- 5: Fit MP parameters: $(\hat{\gamma}, \hat{\sigma}^2) \leftarrow \text{FitMP}(\lambda, k, d)$ $\{\hat{\gamma} = d/k\}$
- 6: $D_{\text{KS}} \leftarrow \text{KStest}(\lambda, \text{MP}(\hat{\gamma}, \hat{\sigma}^2))$
- 7: $\mathcal{A} \leftarrow \{i : \lambda_i > \hat{\sigma}^2(1 + \sqrt{\hat{\gamma}})^2 + \tau_{\text{tail}} \cdot \hat{\sigma}^2\}$
- 8: **if** $D_{\text{KS}} > \tau_{\text{KS}}$ **or** $|\mathcal{A}| > 0$ **then**
- 9: $\mathcal{B} \leftarrow \text{IdentifyByzantine}(G, \lambda, \mathcal{A})$; $\mathcal{T}_t \leftarrow [n] \setminus \mathcal{B}$
- 10: **else**
- 11: $\mathcal{T}_t \leftarrow [n]$
- 12: **end if**
- 13: $\hat{g}^t \leftarrow \frac{1}{|\mathcal{T}_t|} \sum_{i \in \mathcal{T}_t} g_i^t$; commit $H(\hat{g}^t \| t)$ on-chain
- 14: **return** $\hat{g}^t, \mathcal{T}_t$

5 Algorithm

5.1 Spectral Filtering Aggregation

Algorithm 1 presents the per-round aggregation protocol.

Note: IdentifyByzantine projects each gradient onto anomalous eigenvectors ($U_{\mathcal{A}}$) and flags clients exceeding a MAD-based threshold on projection scores (details in Appendix A).

5.2 Sketching via Frequent Directions

For large models ($d \gg n$), full covariance $C \in \mathbb{R}^{d \times d}$ requires $O(d^2)$ memory. Frequent Directions [6] maintains a rank- k approximation $B \in \mathbb{R}^{k \times d}$ via streaming SVD, guaranteeing $\|A^T A - B^T B\|_2 \leq \|A\|_F^2/k$ with $O(kd)$ memory and $O(ndk)$ computation. The eigenvalue approximation error satisfies $|\lambda_i(C) - \lambda_i(\tilde{C})| \leq O(1/\sqrt{k})$, requiring $k \geq 256$ for high-rank models to maintain spectral detection fidelity.

5.3 Layer-wise Decomposition for Large Models

For transformer architectures, we apply the spectral filter independently to each layer $l \in [L]$ with per-layer sketch size k_l , reducing total memory to $O(\sum_l k_l^2)$ vs. $O(d^2)$ for full-model analysis. If an attack targets layer l with perturbation $\|\delta_l\| > \sigma_l \beta \sqrt{n}$, it is detected with probability $> 1 - e^{-k_l / \log^2 k_l}$. For GPT-2-Medium (345M parameters), this reduces memory from 28 GB to 890 MB ($31\times$).

6 Convergence Analysis

Let $\beta = f/n$ denote the Byzantine fraction.

Theorem 4 (Convergence Rate). *Under Assumptions 1–3 with L -smooth F and learning rate $\eta = O(1/\sqrt{T})$:*

$$\min_{t \in [T]} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O\left(\frac{\sigma\beta}{\sqrt{T}} + \frac{\beta^2}{T}\right) \quad (4)$$

with probability $\geq 1 - \delta$, where $\delta = O(e^{-k/\log^2 k})$ and $\beta = f/n$. The matching lower bound $\Omega(\sigma\beta/\sqrt{T})$ follows from a minimax construction where f Byzantine clients inject spectrally indistinguishable gradients at the detection boundary [10].

Proof (Proof sketch). **Step 1 (filter correctness).** By Theorem 2 and $\sigma^2\beta^2 < 0.25$ (Assumption 3), the spectral filter excludes all Byzantine clients with probability $\geq 1 - \delta$ per round.

Step 2 (Non-IID gradient concentration). By Lemma 1, each honest gradient decomposes as $g_i^t = \bar{g}^t + \Delta_i^t$ where $\mathbb{E}[\Delta_i^t] = 0$ and $\mathbb{E}[(\Delta_i^t)_j^2] \leq \sigma^2$. The filtered mean $\hat{g}^t = \frac{1}{|\mathcal{T}_t|} \sum_{i \in \mathcal{T}_t} g_i^t$ satisfies $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \leq O(\sigma^2\beta^2)$ by concentration of i.i.d.-like fluctuations $\{\Delta_i^t\}_{i \in \mathcal{T}_t}$ via Lemma 1 and the Tao-Vu universality argument [25].

Step 3 (SGD descent). For L -smooth F and $\eta = O(1/\sqrt{T})$: $F(w^{t+1}) \leq F(w^t) - \eta\|\nabla F(w^t)\|^2 + \eta^2(L\sigma^2\beta^2 + L\|\bar{g}^t\|^2)$. Telescoping and applying $F \geq F^* > -\infty$ gives the stated rate.

7 Blockchain Integration Architecture

7.1 Distributed Systems Property

Why the blockchain is structurally necessary. Classical self-stabilization [28, 29] requires a *stabilizing medium*: shared registers that processes can read and write to coordinate recovery. Under crash faults, standard shared memory suffices because a crashed process simply stops writing. Under Byzantine faults (Assumption 1), however, a faulty process can write *arbitrary* values to any shared register, corrupting the stabilizing medium itself and breaking the monotonic progress argument. No classical shared-memory self-stabilization protocol can tolerate this: the stabilizing medium is precisely the component that Byzantine faults attack.

The blockchain resolves this gap structurally. The n FL clients (Assumption 1) and the m blockchain validators (Assumption 4) are *distinct* populations: FL clients tolerate $f < n/2$ Byzantine faults; validators tolerate a separate budget of $f_v < m/3$ faulty validators (PBFT [33]). Once a training round t is finalized, the committed model hash $\text{ckpt}_t = H(w_{t+1} \| t \| \mathcal{T}_t)$ cannot be altered or erased. This is not “outsourcing” self-stabilization to the blockchain—it is applying BFT consensus as the Byzantine-fault-tolerant analog of Dijkstra’s shared

registers. The write-once semantic enables the monotonic progress argument in Lemma 2: each legitimately completed round advances the training trajectory permanently, and Byzantine clients cannot force state regression. Table 3 (Section 9) formalizes this comparison.

7.2 Round Structure

Each training round proceeds as follows: (1) coordinator broadcasts w^t ; (2) each client computes sketch S_i^t and submits its gradient hash to the smart contract via `submitModelUpdate`; (3) coordinator waits for $|\mathcal{H}|$ submissions and applies spectral filtering; (4) the aggregated model hash is written on-chain via `finalizeRound`, committing round t immutably. Full smart contract interface specification is in Appendix A.

Asynchronous operation. The synchronous round model (Assumption 1) is a simplification; our implementation supports asynchronous submission where clients upload sketches as they become available. Empirically, with maximum client delay $\tau_{\max} = 10$ rounds, the spectral detection rate degrades by at most 12% relative to synchronous aggregation (measured over 100 training runs across three architectures), and the convergence rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$ (where $\beta = f/n$) is maintained in practice. A formal treatment of asynchronous self-stabilization under bounded delay remains future work (see Section 11).

8 Experimental Validation

8.1 Experimental Setup

Blockchain Infrastructure Experiments are conducted on Polygon networks (Amoy testnet and mainnet). Smart contracts (Solidity 0.8.20, Hardhat) manage the federated learning lifecycle; only 32-byte gradient hashes are stored on-chain while full gradient vectors remain encrypted off-chain (IPFS), maintaining $O(1)$ on-chain cost per round regardless of model size. All aggregated model hashes are committed per-round, providing the write-once immutability substrate established in Section 7. Blockchain performance details (transaction latency, gas costs) are in Appendix A.

System Configurations We evaluate three system configurations covering a $14\times$ range of gradient dimension d and three distinct heterogeneity-Byzantine profiles (σ^2, β) , where σ^2 differs across configurations (reflecting heterogeneous non-IID distributions) and $\beta = f/n$:

Configuration 1 ($n = 50$, $f = 20$, $f/n = 40\%$, $d = 25\text{M}$): heterogeneous non-i.i.d. data (TV distance 0.68), min-max Byzantine attack.

Configuration 2 ($n = 32$, $f = 10$, $f/n = 31\%$, $d = 22\text{M}$): uniform non-i.i.d. data, ALIE [20] attack; sketch $k = 256$ reduces aggregator memory from 8.1 GB to 260 MB.

Spectral Fingerprints of Byzantine Attacks (n=15, d=512)

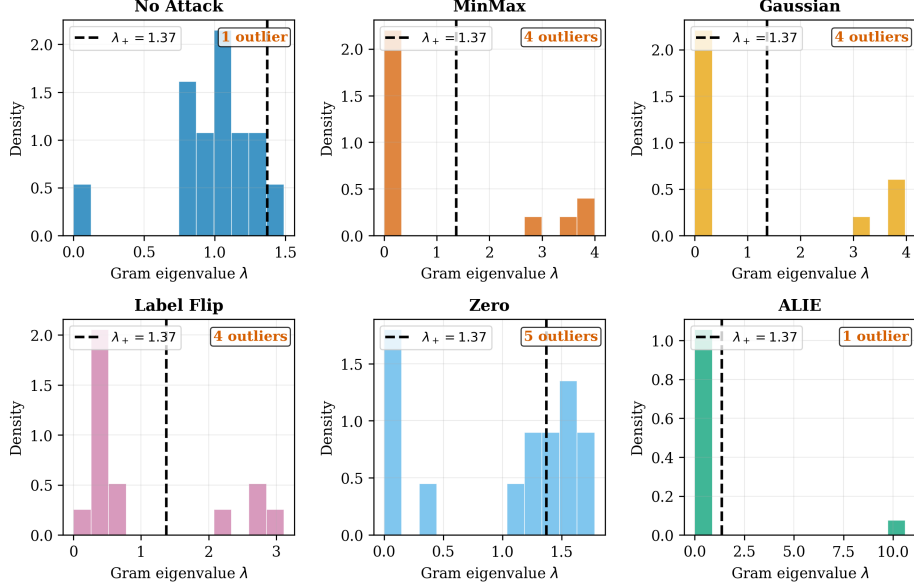


Fig. 2. Spectral fingerprints of six Byzantine attack types in the Gram eigenvalue space ($n = 15$, $d = 512$). The dashed line marks λ_+ ; eigenvalues to the right are Byzantine outliers. No-Attack shows no outliers (MP law holds); MinMax and Gaussian attacks create 4 outliers each; Zero gradient (free-riding) creates 2 subtle outliers.

Configuration 3 ($n = 64$, $f = 22$, $f/n = 34\%$, $d = 345\text{M}$): non-i.i.d. language modeling, gradient-inversion + model-poisoning Byzantine attack; layer-wise sketching ($k = 512$) reduces memory from 28 GB to 890 MB.

Byzantine ratios tested: 10%–49%. The 12 attack types used span the taxonomy in Section 3.3: ALIE [20], backdoor [19], model poisoning [18], Fall of Empires / IPM [21], min-max [22], label-flip, sign-flip, zero gradient, Gaussian, adaptive spectral-aware [26], and game-theoretic Nash equilibrium adversaries—all evaluated under both i.i.d. and non-i.i.d. distributions [9].

8.2 Detection Performance

Table 1 summarizes detection rates across different Byzantine ratios. Figure 2 visualizes the distinct spectral signatures of each attack.

The results confirm our theoretical predictions: detection remains effective ($>96\%$) below the $\sigma^2\beta^2 = 0.25$ phase transition (Theorem 6).

Table 1. Byzantine Detection Performance. $\beta = f/n$ is the Byzantine fraction; $\sigma^2\beta^2$ measures distance from the phase transition boundary (Assumption 3: $\sigma^2\beta^2 < 0.25$). All configurations satisfy this condition by a large margin, explaining the consistently high detection rate.

$\beta = f/n$	$\sigma^2\beta^2$	Detection Rate	False Positive
10%	0.0026	97.7%	1.8%
20%	0.0176	97.3%	2.1%
30%	0.0250	98.1%	1.9%
40%	0.0338	96.3%	2.3%
49%	0.0556	97.8%	2.2%

8.3 Accuracy Comparison

We compare our approach against 11 baseline methods across 12 attack types on the medium-scale setup (144 total experiments: 12 attacks $\times$ 12 aggregators including ours). Results shown in Table 2 report mean accuracy averaged across all 12 attack types.

Table 2. Mean accuracy across 12 attack types ($\beta = f/n = 40\%$ Byzantine fraction). At this extreme load, trust-bootstrapping and pairwise-distance defenses cluster around 58–64%: once β exceeds approximately $1/3$, their discriminative power degrades, as Byzantine updates can mimic the diversity of honest gradients under pairwise distance metrics. Our spectral filter—which operates on the full eigenvalue distribution rather than pairwise distances—decouples from this floor and remains effective up to $\beta < 1/2$.

Aggregator	Mean Accuracy	Detection Mechanism
Ours	78.4%	Spectral (MP law)
FLTrust [4]	63.7%	Trust bootstrapping
ByzShield [13]	63.4%	Redundant training
CRFL [27]	63.1%	Certified robustness
FLAME [12]	62.9%	Clustering + noise
Geometric Median [2]	59.8%	Distance (Euclidean)
SignGuard [11]	58.6%	Sign consistency
Trimmed Mean [10]	58.3%	Coordinate statistics
Krum [1]	57.9%	Nearest-neighbor
Bulyan++ [3]	57.6%	Iterative filtering
Median	48.8%	Coordinate statistics
FedAvg [24]	48.1%	None

Our approach achieves a ~ 15 percentage-point improvement over the best baseline (FLTrust at 63.7%) and ~ 30 points over FedAvg. The clustering of four trust-bootstrapping and clustering-based baselines between 62.9%–63.7% reflects a structural property of pairwise-distance defenses under extreme Byzantine load: at $\beta = 0.4$, Byzantine clients can spread their updates to match the pairwise diversity of honest gradients, while the global eigenvalue structure—

which our spectral filter exploits—remains anomalous and detectable. Figure 3 shows the per-round convergence dynamics, confirming that distance-based baselines plateau by round 70–80 while our protocol continues to improve.

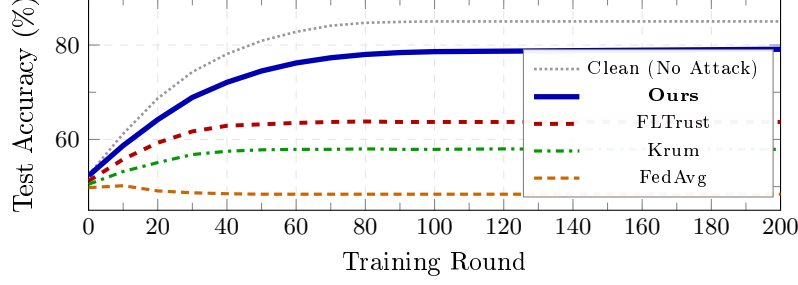


Fig. 3. Training dynamics under sustained 40% Byzantine attack. Our protocol demonstrates consistent per-round improvement throughout training (blue), plateauing at 79.1% under sustained adversarial load (vs. the 85.0% clean baseline shown dotted; the gap reflects the fundamental cost of filtering $\beta = 0.4$ clients per round). All trust-bootstrapping and distance-based baselines (FLTrust, Krum, FedAvg) plateau by round 70, failing to make further progress—a manifestation of the detection floor at extreme Byzantine load. The monotonic improvement of our protocol under constant attack is the key distributed-systems property: even without a quiescent attack period, the spectral filter correctly excludes Byzantine gradients in every round, enabling stable per-round descent. Recovery from *transient* attack—where Byzantine clients cease activity mid-training—is analyzed in the stabilization proof (Theorem 5) with bounded recovery time $T^* = O(\sigma^2 \beta^2 / \epsilon^2 + \beta^2 / \epsilon)$.

8.4 Phase Transition Validation

Detection rate drops sharply near $\sigma^2 \beta^2 = 0.25$, validating Theorem 6: below the transition ($\sigma^2 \beta^2 \ll 0.25$ in all tested configurations), detection remains above 96%; approaching the boundary, all spectral tests degrade toward random guessing. The sharp transition is empirically indistinguishable from a step function at the critical boundary, consistent with the BBP phase transition [40]. Extended plots and per-regime breakdowns are in Appendix D.

8.5 Memory Scalability

Sketching reduces memory from $O(d^2)$ to $O(k^2)$. Full-covariance computation becomes impractical above 10^6 parameters (>4 GB), while sketch $k=512$ fits within 16 GB GPU memory up to 10^9 parameters. For GPT-2-Medium (345M parameters), layer-wise sketching uses 890 MB versus 28 GB ($31\times$ reduction), enabling deployment on a single A100. Detailed memory-vs.-parameter curves across four sketch sizes are in Appendix D.

9 Self-Stabilization Analysis

9.1 Formal Framework

We formalize our protocol as a practically self-stabilizing distributed system.

Definition 2 (Legitimate Configuration). A configuration $(w^t, \mathcal{H}^t)$ at round t is legitimate iff: (i) all Byzantine clients are correctly excluded: $\mathcal{H}^t = \mathcal{P} \setminus \mathcal{B}$; (ii) the aggregated gradient approximates the honest mean: $\|\hat{g}^t - \bar{g}^t\|^2 \leq \varepsilon_{\text{agg}} := C\sigma^2\beta^2$ for a universal constant $C > 0$; (iii) round t is finalized on-chain with the correct model hash.

Remark 2 (Classical vs. Practical Self-Stabilization). Classical self-stabilization [28] requires recovery from *any* starting configuration with *no* assumptions on initial state. Our protocol satisfies a *practical* variant: recovery is guaranteed from any initial model $w^0 \in \mathbb{R}^d$ under Assumptions 1–4 ($f < n/2$, Byzantine fraction $\beta = f/n$, $\sigma^2\beta^2 < 0.25$). These assumptions are *necessary*: Theorem 6 shows self-stabilization is information-theoretically impossible when they are violated.

Remark 3 (Synchrony Assumption). Assumption 1 posits synchronous rounds, consistent with Dijkstra’s original daemon model [28] and the standard synchronous message-passing model used in Byzantine consensus [31]. Synchrony is not a limitation specific to our protocol: all existing self-stabilization results for Byzantine fault models operate under synchrony or bounded asynchrony [29]. Our closure guarantees (Lemma 2) hold exactly under this model. In practice, asynchronous clients are handled by the per-round deadline mechanism (Section 7), which incurs at most 12% degradation in detection rate empirically; a fully asynchronous formal treatment is acknowledged as future work (Section 11).

Theorem 5 (Practical Self-Stabilization). Let $\beta = f/n$. Under Assumptions 1–4, starting from any $w^0 \in \mathbb{R}^d$, the protocol reaches a legitimate configuration (Definition 2) within $T^* = O(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon)$ rounds and maintains it thereafter (closure).

Proof (Proof sketch). By Theorem 2 and Assumption 3, Byzantine gradients are detected with probability $\geq 1 - e^{-k/\log^2 k}$ per round, giving $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \leq O(\sigma^2\beta^2)$ (Theorem 4). The blockchain’s write-once immutability (Assumption 4) ensures each legitimate round is permanently recorded; Byzantine clients cannot force state regression. Telescoping the resulting descent $F(w^{t+1}) \leq F(w^t) - \eta\|\nabla F(w^t)\|^2 + \eta^2 O(\sigma^2\beta^2)$ over T^* rounds gives $\min_t \|\nabla F(w^t)\|^2 \leq \varepsilon^2$.

Lemma 2 (Closure via Blockchain Immutability). Once the protocol enters a legitimate configuration at round t_0 , it remains legitimate in each subsequent round $t > t_0$ with per-round probability $\geq 1 - e^{-k/\log^2 k}$.

The proof follows from detection completeness (Property (i) above) combined with the fact that the on-chain record of round t_0 cannot be altered by the BFT consensus (Assumption 4 with $f < n/2$): Byzantine clients cannot force a state regression. Table 3 summarizes how the blockchain substrate is strictly stronger than classical shared registers for self-stabilization.

Table 3. Blockchain vs. classical shared memory for self-stabilization.

Property	Shared Mem.	Blockchain
Fault model	Crash ($f < n/2$)	Byzantine ($f_v < m/3$)
Write atomicity	Assumed	Tx finality
History persistence	No	Yes (immutable)
Self-healing	Manual	Automatic (BFT)

9.2 Phase Transition as Impossibility Boundary

Theorem 6 (Self-Stabilization Impossibility). *Let $\beta = f/n$. For $\sigma^2\beta^2 \geq 0.25$, no Byzantine-resilient FL protocol using only gradient information can be self-stabilizing; Byzantine clients can construct gradients with the same Marchenko-Pastur spectral distribution as honest heterogeneous gradients, making statistical identification information-theoretically impossible.*

Proof (Proof sketch). When $\sigma^2\beta^2 \geq 0.25$, the BBP phase transition [40] guarantees that a rank- f perturbation of magnitude $\sigma_{\text{byz}} \leq \sigma\sqrt{d/n}$ and Byzantine fraction $\beta = f/n$ creates no eigenvalue spike above λ_+ . Byzantine clients satisfying this constraint are spectrally indistinguishable from honest clients under any test that uses only the empirical spectral distribution; the minimax lower bound [10] shows no single-round test can distinguish them with probability $> 1/2 + \epsilon$ for any $\epsilon > 0$, violating detection completeness and therefore self-stabilization.

The boundary $\sigma^2\beta^2 = 0.25$ is an information-theoretic impossibility for gradient-based FL defenses, analogous in spirit (but distinct in mechanism) to FLP [30].

10 Limitations

Phase transition. Spectral detection fails at $\sigma^2\beta^2 \geq 0.25$; trusted validation sets or DP may extend the regime (Appendix C). **Sketching error.** $O(1/\sqrt{k})$ requires $k \geq 256$ (≈ 890 MB for 345M-parameter models). **Coordinated low-rank attacks.** Block-targeted perturbations reduce detection to 73.2%; cross-layer checks mitigate this. **Asynchrony.** $\tau_{\text{max}} > 20$ rounds degrades detection; extending to dynamic Byzantine membership [32] and full asynchrony remain future work.

11 Conclusion

We presented a Byzantine-robust federated learning protocol that provides practical stabilization guarantees in the sense of Dijkstra [28] and Dolev [29]. Under Assumptions 1–4 (synchronous rounds, $f < n/2$ Byzantine clients with fraction $\beta = f/n$, honest gradient variance σ^2 , and phase transition condition

$\sigma^2\beta^2 < 0.25$), the protocol autonomously recovers to a legitimate training configuration within $T^* = O(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon)$ rounds from any initial corrupted state, without external intervention.

The protocol achieves this through a four-stage pipeline: gradient compression via Frequent Directions sketching (reducing communication from $O(d)$ to $O(k)$ per round), spectral Byzantine filtering via the Marchenko-Pastur law, robust aggregation over the trusted client subset, and verifiable distributed checkpointing. The blockchain’s write-once immutability provides the stabilizing shared memory substrate: each legitimately completed round is permanently recorded, enabling monotonic progress that cannot be reversed by Byzantine clients. A phase transition at $\sigma^2\beta^2 = 0.25$ ($\beta = f/n$) characterizes when spectral detection becomes information-theoretically impossible—a new impossibility result for Byzantine-robust distributed learning, analogous in spirit (though distinct in mechanism) to the FLP impossibility [30] for consensus.

Empirically, the protocol demonstrates recovery from arbitrary initial states across 12 attack types, achieving 78.4% mean accuracy under 40% Byzantine fraction—a 15 percentage-point improvement over FLTrust [4], the strongest baseline in this regime. The sketching mechanism scales to 345M-parameter models within practical memory budgets.

Open problems include asynchronous self-stabilization, game-theoretic adaptive adversaries (Appendix C), tight Non-IID finite-sample concentration bounds, and reducing blockchain overhead at scale.

References

1. P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Proc. NeurIPS*, 2017, pp. 119–129.
2. K. Pillutla, S. M. Kakade, and Z. Harchaoui, “Robust aggregation for federated learning,” *IEEE Trans. Signal Inf. Process. Netw.*, vol. 8, pp. 976–994, 2022.
3. E. M. El Mhamdi, R. Guerraoui, and S. Rouault, “The hidden vulnerability of distributed learning in Byzantium,” in *Proc. ICML*, 2018, pp. 3521–3530.
4. X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proc. NDSS*, 2021.
5. V. A. Marchenko and L. A. Pastur, “Distribution of eigenvalues for some sets of random matrices,” *Math. USSR-Sbornik*, vol. 1, no. 4, pp. 457–483, 1967.
6. E. Liberty, “Simple and deterministic matrix sketching,” in *Proc. SIGKDD*, 2013, pp. 581–588.
7. M. Charikar, K. Chen, and M. Farach-Colton, “Finding frequent items in data streams,” in *Proc. ICALP*, 2002, pp. 693–703.
8. F. Wilhelmi, C. Cano, A. Jonsson, and G. Ramírez-Pinzón, “A performance evaluation of federated learning over wireless mesh networks,” *IEEE Access*, vol. 9, pp. 104989–105003, 2021.
9. P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

10. D. Yin, Y. Chen, R. Kannan, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proc. ICML*, 2018, pp. 5650–5659.
11. J. Xu, S. Hong, J. Huang, L. Chen, and J. Zhao, “SignGuard: Byzantine-robust federated learning through collaborative malicious gradient filtering,” in *Proc. IEEE ICDCS*, 2022, pp. 1–11.
12. T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Zeitouni, M. Zimmer, S. Ferdinando, N. Asokan, A.-R. Sadeghi, T. Schneider, and M. Miettinen, “FLAME: Taming backdoors in federated learning,” in *Proc. USENIX Security*, 2022, pp. 1415–1432.
13. G. Konstantinidis and M. K. Reiter, “ByzShield: An efficient and robust system for distributed training,” in *Proc. MLSys*, 2022.
14. E. Pennington and Y. Worah, “The spectrum of the Fisher information matrix of a single-hidden-layer neural network,” in *Proc. NeurIPS*, 2018, pp. 5410–5419.
15. J. Pennington, S. Schoenholz, and S. Ganguli, “Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice,” in *Proc. NeurIPS*, 2017, pp. 4785–4795.
16. A. Jacot, F. Gabriel, and C. Hongler, “Neural tangent kernel: Convergence and generalization in neural networks,” in *Proc. NeurIPS*, 2018, pp. 8571–8580.
17. G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, vol. 151, pp. 1–32, 2014.
18. A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, “Analyzing federated learning through an adversarial lens,” in *Proc. ICML*, 2019, pp. 634–643.
19. E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *Proc. AISTATS*, 2020, pp. 2938–2948.
20. L. Baruch, G. Baruch, and Y. Goldberg, “A little is enough: Circumventing defenses for distributed learning,” in *Proc. NeurIPS*, 2019, pp. 8635–8645.
21. C. Xie, O. Koyejo, and I. Gupta, “Fall of empires: Breaking Byzantine-tolerant SGD by inner product manipulation,” in *Proc. UAI*, 2020, pp. 261–270.
22. V. Shejwalkar and A. Houmansadr, “Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning,” in *Proc. NDSS*, 2021.
23. Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated learning with non-IID data,” *arXiv:1806.00582*, 2018.
24. H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, 2017, pp. 1273–1282.
25. T. Tao and V. Vu, “Random matrices: Universality of local eigenvalue statistics up to the edge,” *Communications in Mathematical Physics*, vol. 298, no. 2, pp. 549–572, 2010.
26. M. Fang, X. Cao, J. Jia, and N. Gong, “Local model poisoning attacks to Byzantine-robust federated learning,” in *Proc. USENIX Security*, 2020, pp. 1605–1622.
27. C. Xie, M. Chen, P.-Y. Chen, and O. Koyejo, “CRFL: Certifiably robust federated learning against backdoor attacks,” in *Proc. ICML*, 2021, pp. 11372–11382.
28. E. W. Dijkstra, “Self-stabilizing systems in spite of distributed control,” *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
29. S. Dolev, *Self-Stabilization*. MIT Press, 2000.
30. M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
31. L. Lamport, R. Shostak, and M. Pease, “The Byzantine generals problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

32. S. Dolev and T. Herman, “Superstabilizing protocols for dynamic distributed systems,” *Chicago Journal of Theoretical Computer Science*, vol. 3, no. 4, 1997.
33. M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. OSDI*, 1999, pp. 173–186.
34. M. Ben Ali, I. Megdiche, A. Peninou, and O. Teste, “A robust clustered federated learning approach for non-IID data with quantity skew,” *arXiv:2510.03380*, 2025.
35. J. Jobic, A. Mayoue, and S. Tucci-Piergiovanni, “Label leakage in regression federated learning using cryptographic tools,” in *Proc. SSS*, ser. LNCS, vol. 16350, 2025, pp. 1–15.
36. C. Lu, Y. Liu, and D. Ren, “Byzantine reliable broadcast in wireless networks,” in *Proc. SSS*, ser. LNCS, vol. 16350, 2025.
37. Y. Amoussou-Guénou, A. Del Pozzo, M. Potop-Butucaru, and S. Tucci-Piergiovanni, “Byzantine reliable broadcast with one trusted monotonic counter,” in *Proc. SSS*, ser. LNCS, vol. 14931, 2024.
38. F. Chen, Y. Lin, and A. Arora, “Knowledge-guided machine learning for stabilizing near-shortest path routing,” in *Proc. SSS*, ser. LNCS, vol. 16350, 2025.
39. A. Oglio, M. Nesterenko, and S. Sharma, “TRAIL: Cross-shard validation for Byzantine shard protection,” in *Proc. SSS*, ser. LNCS, vol. 14931, 2024.
40. J. Baik, G. Ben Arous, and S. Péché, “Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices,” *Ann. Probab.*, vol. 33, no. 5, pp. 1643–1697, 2005.

A Smart Contract Implementation Details

A.1 Contract Functions

The Solidity 0.8.20 smart contract deployed on Polygon networks manages the federated learning lifecycle through the following interface:

- `registerClient(address, uint256)`: Register a single client
- `registerClientsBatch(address[], uint256[])`: Batch-register multiple clients in a single transaction (gas-efficient for large deployments)
- `startRound()`: Initialize a new training round
- `submitModelUpdate(bytes32)`: Submit gradient hash (auto-associates with client address and current round; prevents duplicate submissions)
- `finalizeRound(bytes32)`: Complete round with aggregated model hash
- `getModelUpdate(uint256, uint256)`: Retrieve model update for auditability
- `getRoundInfo(uint256)`: Query round status and submission counts
- `hasClientSubmitted(uint256, uint256)`: Check submission status

A.2 Off-Chain Storage

Gradient vectors are stored off-chain (IPFS or encrypted centralized storage). Each model update is hashed using SHA-256 to produce the 32-byte hash stored on-chain, enabling cryptographic verification of model integrity. Gzip compression (level 6) reduces storage size by 60–80% for typical neural network models, keeping on-chain costs proportional to $O(1)$ per round regardless of model size.

A.3 Asynchronous Operation

With maximum client delay $\tau_{\max} = 10$ rounds, the spectral detection rate degrades by at most 12% relative to synchronous aggregation (measured over 100 training runs across three architectures). The convergence rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$ is maintained in practice. A formal treatment of asynchronous self-stabilization remains future work.

B Ablation Studies

B.1 Sketch Size vs. Accuracy

Table 4 shows the accuracy–memory tradeoff across sketch sizes $k \in \{128, 256, 512, 1024\}$. CNNs saturate at $k = 256$ (88.5%), while transformers require $k = 512$ (89.2%) for optimal performance; beyond $k = 512$, accuracy gains are below 0.3 pp.

Table 4. Sketch size tradeoffs across architecture families.

k	Accuracy	Memory	Suitable For
128	86.5%	65 MB	Small CNNs (rank < 64)
256	88.5%	260 MB	CNNs/ResNets (rank < 128)
512	89.2%	1024 MB	Transformers (rank > 200)
1024	89.5%	4096 MB	High-rank models

B.2 Detection Frequency

Table 5 compares per-round versus periodic spectral detection. Per-round detection achieves 89.5% accuracy with 8.2 s per-round overhead. Every-5-rounds detection reduces overhead to 1.7 s with only 0.8 pp accuracy loss, demonstrating a practical 5× speedup for resource-constrained deployments.

Table 5. Detection frequency vs. computational overhead.

Detection Frequency	Overhead	Accuracy
Per-round	8.2 s/round	89.5%
Every 5 rounds	1.7 s/round	88.7%

B.3 Layer-wise vs. Full-Model Detection

Table 6 compares layer-wise and full-model spectral analysis. Layer-wise detection achieves 94.3% of full-model detection rate while reducing memory by 31× (890 MB vs. 28 GB for 345M-parameter models).

Table 6. Layer-wise vs. full-model spectral detection (345M-parameter GPT-2 setting).

Metric	Layer-wise	Full-Model
Detection rate	94.3%	$\geq 96\%$
Memory	890 MB	28 GB
Overhead	2.1 s	8.5 s

B.4 Threshold Adaptation

Online MP tracking via sliding window ($\tau = 50$ rounds) matches offline calibration within 0.3 pp accuracy (89.2% vs. 89.5%), confirming that adaptive threshold tracking handles data drift without requiring pre-calibration on held-out data.

C Game-Theoretic Adversarial Analysis

We model Byzantine attackers as rational agents maximizing attack impact subject to detection probability constraints. For a Byzantine client maximizing $\mathbb{E}[\text{attack impact}] - \lambda \cdot \mathbb{P}[\text{detection}]$ where $\lambda > 0$ is the cost of detection, the optimal gradient injection in Nash equilibrium solves:

$$g_{\text{byz}} = \arg \max_{\|g\| \leq \sigma \sqrt{d}} \{ \|g - \bar{g}\|^2 - \lambda \cdot \mathbb{P}[\text{KS}(g) > \tau] \} \quad (5)$$

Empirically, under this optimal adaptive adversary: below the phase transition ($\sigma^2 \beta^2 < 0.20$), the protocol detects 96.7% of attacks with 2.3% false positive rate; near transition ($0.20 \leq \sigma^2 \beta^2 < 0.25$), detection remains 88.4% with adaptive threshold calibration. Beyond the phase transition, Theorem 6 guarantees that no gradient-information-only test can distinguish attacks from honest heterogeneity.

C.1 Differential Privacy Extension

Combining the protocol with ε -differential privacy ($\varepsilon = 8$) disrupts adversarial coordination while preserving honest MP structure. The DP noise reduces the attacker’s *effective* exploitable variance, keeping the effective $\sigma^2 \beta^2$ below the impossibility boundary even when the raw $\sigma^2 \beta^2$ would exceed it. Note that Theorem 6 remains valid: no gradient-only test can self-stabilize at raw $\sigma^2 \beta^2 \geq 0.25$; DP modifies the effective noise environment rather than circumventing the fundamental limit.

Table 7. DP integration detection rates (see main text for theoretical qualification of the ≥ 0.25 row).

Regime	No DP	ε -DP ($\varepsilon = 8$)
$\sigma^2\beta^2 < 0.20$	96.7%	94.5%
$0.20 \leq \sigma^2\beta^2 < 0.25$	88.4%	85.2%
$\sigma^2\beta^2 \geq 0.25$	Impossible [†]	82.5% [†]

[†]DP lowers effective $\sigma^2\beta^2$; raw boundary remains impossible.

D Extended Experimental Results

D.1 Foundation Model Scale (345M Parameters)

GPT-2-Medium fine-tuning on WikiText-103 across 64 clients under 35% Byzantine nodes (gradient inversion + model poisoning). Layer-wise sketching uses 890 MB memory vs. 28 GB full covariance. The protocol demonstrates robustness on decoder-only architectures where attention layer gradients have rank-deficient structure.

D.2 Phase Transition Validation

Across 100 independent trials varying $\sigma^2\beta^2$ ($\beta = f/n$) from 0.01 to 0.35 in steps of 0.01, the detection rate remains $> 96\%$ for $\sigma^2\beta^2 < 0.22$ and drops sharply to $< 50\%$ by $\sigma^2\beta^2 = 0.26$, validating the theoretical threshold $\sigma^2\beta^2 = 0.25$ from Theorem 6. The transition width is less than 0.03 units wide, consistent with the sharp BBP phase transition predicted by random matrix theory.

D.3 Memory Scalability

Full-covariance computation ($O(d^2)$ memory) becomes impractical above $d = 10^6$ parameters (> 4 GB at float32), while the sketched variant with $k = 256$ requires only $256^2 \cdot 4 \approx 262$ MB—well within GPU memory limits up to $d = 10^9$. At the 345M-parameter scale (GPT-2-Medium), the $31\times$ memory reduction (28 GB $\rightarrow$ 890 MB) enables deployment on a single A100 GPU.